NLP Based Resume Analysis and Ranking System

Aabha Bhide
Computer Science and Engineering(AI&ML)
A.P.Shah Institute of Technology
Thane, India
aabhabhide28@gmail.com

Vijaya Bharathi Jagan
Computer Science and Engineering(AI&ML)
A.P.Shah Institute of Technology
Thane, India
vbjagan@apsit.edu.in

Aditi Gadhave
Computer Science and Engineering(AI&ML)
A.P.Shah Institute of Technology
Thane, India
aditigadhave77@apsit.edu.in

Priyanka Patil
Computer Science and Engineering(AI&ML)
A.P.Shah Institute of Technology
Thane, India
pjpatil@apsit.edu.in

Niharika Bandekar
Computer Science and Engineering (AI&ML)
A.P.Shah Institute of Technology
Thane, India
niharikabandekar124@apsit.edu.in

Sumeet Gupta
Computer Science and Engineering (AI&ML)
A.P.Shah Institute of Technology
Thane, India
sumeetgupta69@apsit.edu.in

Abstract **-** Manual resume screening and keyword-based applicant tracking systems do not capture semantic relevance, provide actionable feedback and support interpretable candidate ranking. This paper presents an NLP-based resume analysis and ranking system designed for student self-assessment and recruiter decision support. The system computes a reproducible weighted Applicant Tracking System (ATS) score by combining Term Frequency–Inverse Document Frequency (TF-IDF) cosine similarity, sentence-transformer–based experience matching, rule-based formatting evaluation, explicit skill overlap and education alignment. A hybrid symbolic–semantic skill gap module extends job skills using an ontology and applies zero-shot natural language inference to estimate missing skill confidence. Recruiter-side candidate ranking adds structured candidate features with semantic similarity using a lightweight neural ranking model, supported by SHapley Additive exPlanations (SHAP)-based post-hoc explainability for top-k candidates. Implemented using FastAPI, Facebook AI Similarity Search (FAISS) and SQLite, the core ATS and ranking modules run locally with reproducible outcomes. The system supports SDG 4 (Quality Education) and SDG 8 (Decent Work and Economic Growth) through interpretable and reproducible hiring.

Keywords: Applicant Tracking System (ATS), Semantic similarity, Skill gap detection, Ontology-based skill mapping, Zero-shot natural language inference, SHapley Additive exPlanations (SHAP) explainability, FastAPI, Facebook AI Similarity Search (FAISS), SQLite

I. INTRODUCTION

In the modern job market, recruiters receive a mountain of resumes for every open role, making manual screening impractical, time consuming and ripe for human error. Conventional applicant tracking systems depend majorly on keyword matching, which fails to capture semantic context, often giving inaccurate final results. Solving this issue is important to increase hiring efficiency, reduce bias and make the process more transparent. Recruiters face challenges like increased cost and delay due to infective screening of candidates, while the candidates face issues like lack of clarity on criteria for evaluation and areas for improvement. Thus, an intelligent and explainable approach is essential.

To solve this problem, an NLP-based Resume Analysis and Ranking System handles resume screening using NLP, machine learning and Advanced deep learning. This system uses TF-IDF with cosine similarity for keyword matching, Sentence Transformer (MiniLM) embeddings with FAISS for semantic similarity, JobBERT for skill extraction and BART-based Natural Language Inference for skill-entailment analysis to produce explainable ATS scores and skill-gap insights. Implemented using a FastAPI backend, React with Chakra UI frontend and SQLite database, the system delivers a lightweight, efficient, secure and transparent platform for both recruiters and candidates. In this paper, design and implementation of an NLP-based resume analysis and ranking system is presented. It mentions the problem motivation, system architecture and methodology used for resume analysis and candidate ranking and also describes system functionality via architectural diagrams.

II. RELATED WORKS

Initial studies focused on using machine learning for candidate ranking based on resume features [8]. But these approaches relied on structured inputs and struggled with unstructured resume data. This gap was later mentioned in studies that introduced NLP based resume parsing techniques to extract structured entities such as skills and experience from unstructured documents [2]. To go beyond keyword-based matching, semantic similarity methods were introduced. Use of Sentence-BERT–based models improved candidate-job alignment by allowing contextual matching between the resumes and job descriptions [3].Later ethical and operational concerns were also studied. Explainable AI was emphasized to reduce bias and enhance transparency in hiring process [5], whereas secure recruitment systems focused on authentication and access control to protect sensitive data [6]. Newer studies shifted to candidate retrieval using ML-driven search and ranking models to deal with thousands of applications. [1]. While these studies address key pieces of automated hiring, they often function independently.

In summary, the current work contributes valuable solutions to individual stages of the hiring pipeline but lacks an integrated framework that combines semantic matching, skill validation, explainable ranking, analytics and security. The proposed system addresses this gap through an end-to-end, transparent and scalable recruitment architecture.

III. METHODOLOGY

This section of the paper presents the methodology implemented for the proposed NLP-Based Resume Analysis and Ranking System. The system supports two user roles, Student and HR through their respective web portals. It is a modular and layered architecture which allows automated resume screening, semantic job-candidate matching, explainable ranking and personalised feedback. The methodology section explains the general workflow, core system architecture and functioning of each module on both the sides, Student and HR. Figure 1 represents the system context of the resume analysis system, highlighting the date exchange taking place between students, recruiters and the core system, excluding the internal module-level details.

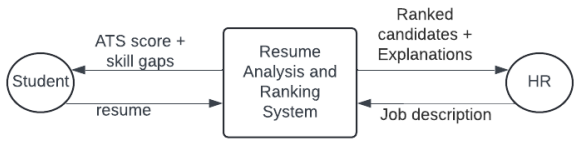

Fig. 1. System Context Diagram

A. System Overview

The proposed system automates candidate evaluation by using Natural Language Processing (NLP) and Machine Learning (ML) techniques to analyse resumes against job description. Students receive a detailed ATS report along with the feedback for improvement, while HR users receive ranked candidate backend along with a locally deployed AI engine, thus ensuring data privacy, real-time inference and scalability. Figure 2 represents the overview of the system.

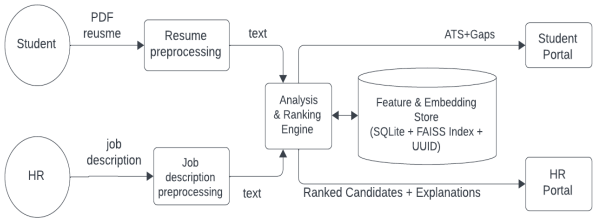

Fig. 2. Functional Architecture Diagram

B. Module-Specific Implementation

I) Student Modules:

Job Browsing & Application Module:

The Job Browsing and Application Module exposes a RESTful interface integrating a keyword-based filtering, optional semantic relevance enhancement and transactional application submission. Job discovery requests query a relational schema indexed on job status and text-search (tsvector) fields, by providing a paginated and filter aware result sets. Authenticated application submission is then processed through an idempotent write path enforced by a composite unique constraint, thus preventing duplicate applications. While submitting, an ATS compatibility score is computed synchronously, the application record is persisted within an ACID-compliant transaction and a Redis stream event is emitted enabling downstream analytics and monitoring. The module is designed to preserve transactional integrity and auditability.

Resume Ingestion and Parsing Module:

The module handles the acquisition, normalization and structured storage of resume data. PDF and DOCX files are accepted making use of PyPDF2 and python-docx. Text extracted is pre-processed including removal of extra whitespaces, converting to lowercase, normalizing Unicode characters (Normalization Form Compatibility Decomposition-NKFD) and eliminating special characters except . , ; : ! ? ( ) [ ] / @. Rule-based section segmentation using predefined regex patterns is used to identify the Education, Experience, Skills, Projects, Certifications, Languages and Interests sections while storing each section separately. Preliminary skill extraction is performed through keyword matching against a curated skill-words list. The original files and processed text are later stored in the SQLite table with ISO 8601-formatted timestamps.

Job-Match Analysis Module:

The module calculates the compatibility between a student's resume and a target job description by computing a weighted Applicant Tracking System (ATS) score. It evaluates five independent resume attributes namely keyword relevance, resume formatting quality, experience relevance, skill matching and educational alignment. Keyword relevance is given the highest weight of 30% to capture lexical correspondence between the resume and job description. Experience relevance is assigned 25%, measuring the semantic similarity between resume experience sections and job requirements. Resume formatting quality contributes 20% to ensure ATS-friendly structure and readability. Skill matching weighs 15% by evaluating explicit alignment with required job skills, while educational alignment holds 10% of the remaining weight to confirm degree-level eligibility. Each component score is later normalized and combined by making use of a fixed weighted aggregation scheme thus ensuring interpretability, consistency and deterministic evaluation without having to rely on external large language model APIs.

Let the normalized component scores be defined as follows:

K: Keyword relevance score

F: Resume formatting score

E: Experience semantic similarity score

S: Skill match score

D: Education match score

Keyword Relevance Score (TF-IDF Cosine Similarity)-

The keyword score is computed using TF-IDF vectorization followed by cosine similarity between the resume text $R$and the job keyword document J:

$$K = \cos(\mathbf{v}_R, \mathbf{v}_J)$$

where $\mathbf{v}_R$ and $\mathbf{v}_J$ denote TF-IDF vectors constructed over unigrams and bigrams.

Formatting Score (Heuristic Aggregation)-

Resume formatting quality is assessed using rule-based heuristics capturing structural compliance:

$$F = \frac{H + B + L}{3}$$

where,Hindicates the presence of required section headers, B denotes bullet-point density and L represents compliance with an optimal resume length range.

Experience Match Score (Semantic Similarity)-

The resume is split into overlapping text chunks $\{r_i\}$. Each chunk and the job description $J$are embedded into a shared vector space using a sentence-transformer model. The experience score is defined as the maximum cosine similarity:

$$E = max_i \cos(\mathbf{e}_{r_i}, \mathbf{e}_J)$$

This captures the strongest semantic alignment between any resume experience segment and the job requirements.

Skill Match Score-

Explicit skill overlap is computed as:

$$S = \frac{|S_r \cap S_j|}{|S_j|}$$

where S_r represents the set of skills extracted from the resume and $S_j$represents the required job skills.

Education Match Score-

Educational alignment is modelled as a binary score:

$$D = \begin{cases} 1, & \text{if a required degree keyword is present} \\ 0, & \text{otherwise} \end{cases}$$

Final ATS Score-

The overall ATS score is computed using fixed weights that sum to unity:

$$\text{ATS}_{score} = 0.30K + 0.20F + 0.25E + 0.15S + 0.10D$$

$$\text{ATS}_{\%} = \text{ATS}_{score} \times 100$$

Example:

Let the job description for a Junior AI Engineer be as follows:

Location: Mumbai (Hybrid)

Experience Level: Entry-Level (0–1 year)

Employment Type: Full-time

Required Skills: Python, Machine Learning, TensorFlow, Scikit-learn, OpenCV, SQL, Git, Problem Solving, Team Collaboration

Preferred Education: Bachelor of Engineering (B.E.) in Computer Science, Artificial Intelligence, or related field

Experience Expectation: Full-time, in-person internship or project experience in AI/ML

Let, the following resume be tested against the above mentioned job description:

Final-year B.E. in Computer Science & Engineering (AI & ML specialization) at APSIT, Mumbai University. Aspiring AI & ML enthusiast with practical experience from

internships and academic projects, passionate about applying technology to solve real-world problems.

Skills**:** Python, Java, C, JavaScript, PHP, HTML, CSS, JS, PHP, MySQL, Scikit-learn, TensorFlow, Keras, CNN, Regression models, AWS (EC2, S3, Data Pipelines), ETL, SQL, OpenCV, Pandas, NumPy, Matplotlib, Git, Jupyter Notebook

Education: B.E. in Computer Science & Engineering (AI & ML), APSIT, Mumbai University (2021–Present, entering 7th semester).

Calculating the ATS Score for the above resume, against the given job description:

Keyword Relevance (K): TF-IDF cosine (resume, job_keywords) = 0.80

Formatting Score (F): (Header present + bullet density (0.85) + resume length (400–800)) / 3 = 0.90

Experience Relevance (E): max_i cos (MiniLM(chunk_i), MiniLM(job)) = 0.75

Skill Match (S): | {Python, ML, TensorFlow, …} ∩ required| / |required| = 0.75

Education Match (D): Degree keyword "B.E." found, D = 1.0

Final ATS Score:

ATS_{score} = (0.30·0.80+0.20·0.90+0.25·0.75+0.15·0.75+0.10·1.0) = 0.8

$ATS_{\%}$ = 0.8 x 100 = 80%

Explainability and Feedback Layer-

To ensure transparency, the module converts deterministic component scores (K, F, E, S, D) into qualitative feedback. A rule-based format checker flags sub-optimal bullet usage, missing section headers and length violations, whereas a zero-shot Natural Language Inference (NLI) classifier calculates the confidence scores for missing job-relevant skills. These calculations are translated into structured suggestions like "add measurable outcomes", "insert a dedicated Skills section" and is then displayed along with the ATS percentage, thus helping students to improve their resume before the ranking and recruiter-side evaluation is performed without changing the underlying scoring formulation.

Rule-Based and Semantic Diagnostics

Student-facing explainability problem is accomplished without exposing model-level attribution techniques. Instead, deterministic scoring components are read using rule-based heuristics and semantic diagnostics to produce human-readable feedback. Thus, clarity and usability is prioritized while still preserving the reproducibility of the underlying evaluation pipeline.

Feedback Synthesis and Presentation-

Instead of giving raw numerical scores or low-level model internals, the system presents structured qualitative feedback. This includes area for improvement (e.g.: missing project sections, unquantified achievements), actionable improvement suggestions (e.g., adding measurable outcomes or explicit skill listings) and skill deficiencies along with the confidence indicators.

Skill Gap Module:

The module assesses a student's resume to identify job-relevant skills that are present as well as missing with respect to a given role. The module first expands the set of required skills using a predefined skill ontology to account for hierarchical parent–child relationships. Explicit skill presence is then detected through case-insensitive matching between the resume text and the expanded skill set. Skills not detected in the resume are further evaluated using a zero-shot natural language inference classifier, which estimates the confidence that the resume does not semantically express each missing skill. This hybrid symbolic–semantic approach enables interpretable and training-free skill gap detection, producing a structured output consisting of matched skills, missing skills and confidence scores for each identified gap.

To provide actionable help apart from skill gap analysis, the system provides an optional course recommendation layer which is performed strictly after the skill gap inference and a mock interview model.

Course recommendation module: For every missing skill that exceeds a predefined threshold limit, the module queries an external large language module-based recommendation service to recover upskilling resources.

Mock Interview module: To improve the coverage and role specificity apart from the static interview questions, this module makes use of an external large language model for augmenting interview questions. For the given job roles and skill profiles, the system queries the external model to produce additional role-specific interview question bank, which is subsequently filtered and embedded with the use of same sentence-transformer model used before retrieval.

Both these optional modules are architecturally dissociated from the core functions of the system and does not change the core detection of the present skills, confidence score estimation or ATS score calculation or the similarity computation. Since both the modules make use of external API, they are non-deterministic and replaceable, thus ensuring reproducibility and auditability among the core analytical modules.

Application Tracking & Progress Monitoring Module:

The module collects student interactions and evaluation outputs and gives a combined view of resume usage and application progress. It calculates summary statistics which includes resume and application counts, interview status transitions and average ATS performance. Application progress is summarized using a status-wise distribution derived from distinct application state transitions. All metrics are calculated by using deterministic aggregation functions over locally stored relational data, which ensures transparency, low-latency analytics and reproducibility.

II) HR facing module

Job Posting Management Module:

The module delivers a secure RESTful interface for establishing and managing job vacancies. A FastAPI router verifies incoming job data against a Pydantic model and implements role-based access control using JWT verification, restricting job creation to HR users. Validated postings are persisted using ACID-compliant database operations, with consistency upheld through schema-level constraints, including CHECK conditions enforce positive values and min_salary ≤ max_salary and categorized job status options. Each posting is linked with the user's HR profile to ensure their ownership and modification. An optional issuing flag enabling instant enablement of the opening triggers an event-based notification through Redis streams, allowing semantic indexing parts to refresh job encodings asynchronously. This design assures coherence, security and punctual accessibility of job data for suggestion and corresponding workflows.

Application & Interview Management Module:

The module administers the complete cycle of job entries using role-based APIs, a reliable finite-state device and exchange-safe retention, ensuring accuracy, accountability and actual-time system promptness.

Students send applications through POST /jobs/{job_id}/apply. This method prevents replicate requests by implementing a distinct composite condition on (student_profile_id, job_id) at the database tier. Upon confirmation, the device calculates the ATS score internal utilizing the resume–role aligning component and adds a new log in the Submissions table with a tart status of SUBMITTED. The process is created to provide fast response which is apt for reactive user experience.

Deterministic State Transition Model

Each administration moves through a finite-state machine (FSM) whose status is matching the router-level enumeration:

$$S = \{\text{SUBMITTED}, \text{SHORTLISTED}, \text{INTERVIEW_SCHEDULED}, \text{INTERVIEWED}, \text{OFFERED}, \text{REJECTED}, \text{WITHDRAWN}\}$$

and action set

A= {submit, shortlist, schedule_interview, record_interview, offer, reject, withdraw}

OFFERED, REJECTED and WITHDRAWN are the final pointers; no further changes allowed.

State changes are done through PATCH /applications/{application_id}/status, open solely to HR-role users. A partial transition operation,

$$\delta: S \times A \rightarrow S$$

is enforced at the service layer such that only workflow-legal transitions are accepted. Invalid transitions (e.g., REJECTED → OFFERED) are rejected, guaranteeing deterministic and valid application progression.

Interview Scheduling Semantics

Converting an entry to INTERVIEW_SCHEDULED sets the state only; date, time and mode are saved subsequently by means of a follow-up POST/applications/{id}/interview-details API point, preserving the central flow separated.

Access Control and Data Isolation-

Row-level access control guarantees HR users can act only on entries of their job listings, implemented by the requirement:

application.job_description.hr_profile_id = current_user.hr_profile.id

This assures recruiter ownership and restricts inter-vacancy conflict.

Audit Logging and Event Propagation-

All the state changes are fixatedly audit-recorded, recording the last state, current status, operating user ID and time stamp. Each successful mutation emits a structured event (application_status_changed) to a Redis Stream consumed via user groups for resilience.

HR Dashboard and Progress Visualization-

A paged GET /applications API enables recruiter dashboards, facilitating sorting by ATS score or entry submission time. The application progress signal displayed in the HR shell presents a phase-based proxy coordinated with FSM phases, rather than an absolute gross total metric.

Bulk Resume Upload Module:

The module allows HR to process several candidates resumes in one action. The app stores ZIP archives having several PDF or DOCX files, extracts text data using PyPDF2 and python-docx and employs the same reliable ATS scoring workflow used for each and every resume analysis. To avert duplication processing, an encrypted

hash is calculated for each & every resume and measured against current records to spot redundant entries.

The retrieved text and related metadata are continuously stored beside the original data, allowing optimized recycle by backend analysis and ranking functions. In initial testing with batch loads, the system displayed a practically observed average computing time of 4.5s per resume, refinement for big-scale databases is saved for later work.

Candidate Ranking & Explainability Module:

After the resume upload and ATS-based analysis, shortlisted candidates are graded using a composite learning-based model enhanced with retrospective understanding. The goal of this function is to make a stable, clear ranked list of candidates/students while still keeping recruiters trust and decision clarity

Each candidate is encoded as a fixed-length numeric feature vector:

$$x_c = \begin{bmatrix} ATS_c, \\ YearsExp_{c,} \\ SkillMatch_{c,} \\ EduLevel_c, \\ LocationMatch_{c,} \\ SalaryFit_c \end{bmatrix}$$

where ATS_c denotes the deterministic resume–job compatibility score, $YearsExp_c$ represents years of professional experience, $SkillMatch_c \in [0,1]$ is the proportion of required skills matched, $EduLevel_c$ is an ordinal encoding of education level, $LocationMatch_c \in \{0,1\}$ indicates geographic compatibility and SalaryFit_c models the alignment between the candidate's expected salary s_c and the job budget b. SalaryFit_c is clipped to [0,1] to ensure bounded values. All features are standardized using precomputed mean and variance parameters to ensure numerical stability.

Candidate relevance is estimated using a lightweight feed-forward neural network (RankNet) function. The network outputs a scalar relevance score for each candidate. Model parameters are pre-trained and loaded in evaluation mode at inference time, ensuring deterministic and reproducible ranking behaviour.

To incorporate semantic alignment beyond structured attributes, cosine similarity is computed between the job description and candidate resume text using sentence-level embeddings. FAISS is employed for efficient similarity computation. The final ranking score is obtained via a weighted ensemble:

$$Score_c = 0.7 \cdot f(x_c) + 0.3 \cdot s_c^{sem}$$

where s_c^{sem} denotes the semantic similarity score. Candidates are ranked in descending order of this combined score.

Explainability via SHAP-

SHAP (SHapley Additive exPlanations) is applied exclusively within the HR Candidate Ranking module to provide transparent, recruiter-facing explanations of the final relevance score. For each top-ranked candidate, SHAP DeepExplainer computes per-feature attributions that quantify the influence of factors such as ATS score, years of experience, skill-match ratio, education level and related attributes on the ranking outcome. Even though SHAP credit values are formed internally in the HR Candidate Ranking function, the current HR portal doesn't touch these outputs graphically. The module outputs an ordered set of candidates, where each entry includes an overall relevance metric, a recommendation category (Highly Recommended / Recommended / Not Recommended) and human-readable, feature-level interpretability details. All processing is executed locally with no dependency on external APIs, which supports low-latency performance, consistent results and strong data protection.

IV. RESULTS

System Interface and Module Results

a) Student Portal

Resume Job-Match Insights: When each Resume is uploaded, it is immediately followed by a 0-100% ATS Score that is segregated into five colour-coded bars: keyword, skill, format, education and experience. Students get suggestions like "Missing Python skill detected" or "Experience contributed positively", helping students in a self-improving support system that guides and coaches them for revision without external coaching. The score is cloned using the same TF_IDF, rule-based pipeline, MiniLM, ensuring full inspection.

Skill-Gap Discovery: The matched vs. missing skills presented by a zero-shot entailment classifier as clickable chips are marked up with confidence badges; a more extensive ontology layer (e.g., React → JavaScript) prevents synonyms' blind side. The result prioritises gap list links to a course catalogue (present locally but queried via an external API), while maintaining training-free, cloud-free skill detection, it transforms the diagnostic result into an actionable learning path.

Resume Analysis: The local hybrid pipeline of 5.B.III instantly renders the student side analysis screen by rendering the deterministic ATS Score (0-10%), it then decomposes the score into colour-coded "missing /matched skills" chips that directly reflect the ontology-expanded, zero-shot NLI results from the Skill Gap Module. within the same MiniLM + SQLite stack, every improvement bullet and confidence badge is generated solely from the rule-based and semantic diagnostics, ensuring the whole feedback experience remains cloud-free, reproducible and fully aligned with the methodological accuracy.

Semantic Job Recommendations: The "Jobs You Might Like" recommends scrollable job cards, automatically matching the student's profile, omitting explicit search items. This uses cosine similarity, which helps in context-aware discovery and encourages exploratory browsing. Local MiniLM encoder and FAISS flat index are used to compute all similarities.

Mock-Interview Generation: A local FAISS index primarily retrieves interview questions over a curated question bank by combining the resume and preferred job types as a query. An external LLM, i.e. Large Language Model API, can optionally generate additional role-aligned questions to enhance coverage. The same sentence-Transformer filters and embeds the newly generated questions. Since the augmentation layer is built separately, not as tightly connected, the core semantic retrieval pipeline remains reproducible, methodologically consistent and auditable.

Activity Timeline & Self-Monitoring: A chronological list of events records every status change so nothing is lost (submitted → shortlisted → interviewed →offer→ Accepted) to local ISO-8601 timestamps. The progress bar converts discrete states into perceived advancement, making the user feel the progress and stay motivated, which improves engagement without predictive modelling or cloud telemetry.

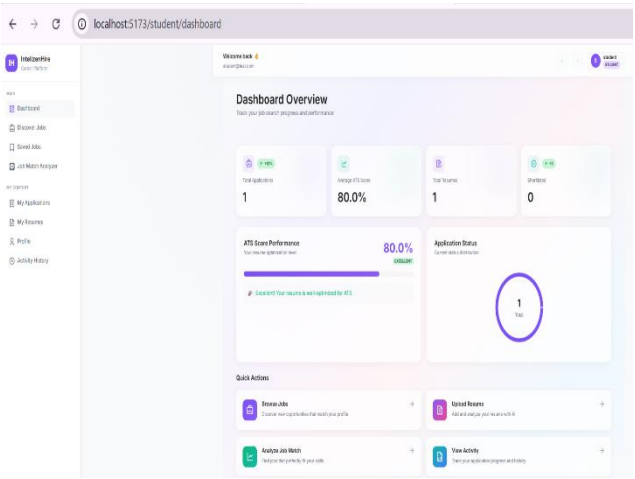

Fig. 3. Student Dashboard

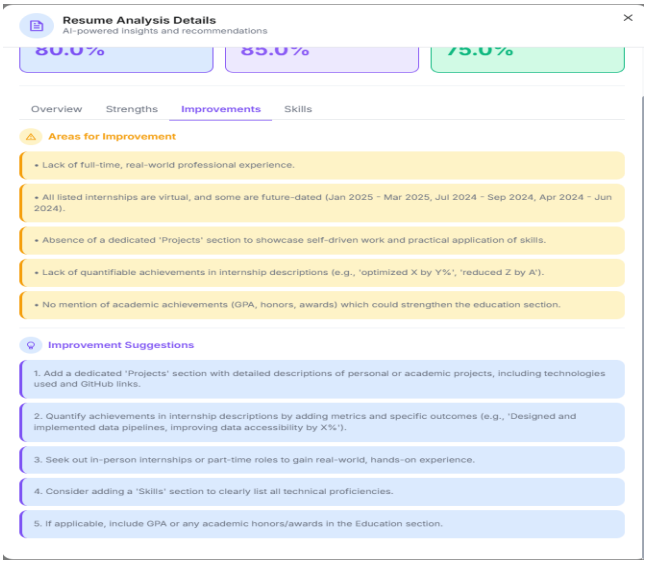

Fig. 4. Descriptive resume analysis

.b) HR Portal

Bulk-Upload & Analysis: Uploading multiple PDF resumes by dragging Triggers the local PyPDF2 + MiniLM pipeline, which yields instant ATS score bars and a rule-based ranked list produced with the help of the RankNet + SHAP ensemble; the local relevance score helps to obtain the "Not Recommended" badge with no external ranking service consulted.

Application Pipeline: The finite-state-machine transition triggered by the PATCH endpoint is reflected by the 'INTERVIEW SCHEDULED' chip; the SQLite Applications table stores timestamp and ATS score locally, so no cloud logging is needed, which ensures full auditability and each transition is permanently logged and propagated via Redis streams.

Job Posting Management: The 'Create New Job' form gets filled automatically with skills, degrees and company names using the JobBERT NER extractor applied locally to the free-text job description. A Redis event is activated with the 'Publish immediately' toggle, which triggers a local FAISS job index rebuild, keeping synchronised and cloud-free semantic search and recommendation feeds.

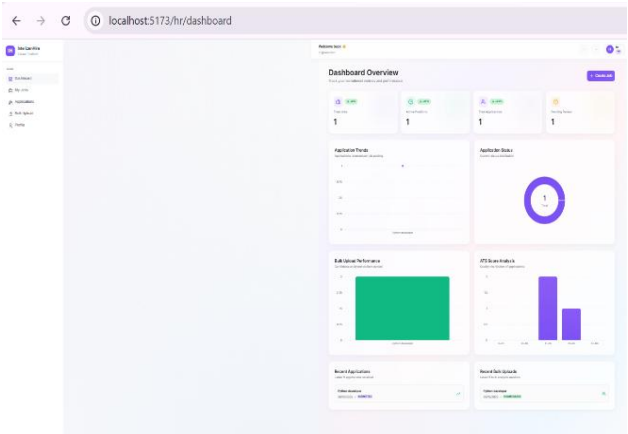

Fig. 5. HR Dashboard

V. Conclusion

This paper presents an NLP-based Resume Analysis and Ranking System which goes beyond the traditional keyword matching approach used for automating recruitment. The system provides accurate and transparent candidate–job matching by integrating semantic similarity modelling, skill gap analysis and interpretable ATS scoring within a modular architecture. Further inclusion of dedicated dashboards for both, recruiters and candidates enhance usability and decision-making. A major limitation of this system is its dependency on textual resume data and predefined scoring weights, which may need to be personalized for various hiring contexts. Future work focuses on integrating additional data sources like assessments, project portfolios and interview feedback with adaptive scoring mechanisms and bias-aware evaluation. The system can also be expanded further as an intelligent interview recommendation and analytics modules, as well as cloud-based deployment, which could support large-scale and real-time recruitment workflows.

VI. References

[1] Wen Li and XiuKao Zhou, Machine Learning-Based HR Management Information Retrieval and Classification Algorithm, Scalable Computing: Practice and Experience, Vol. 25, No. 6, pp. 5431-5440, 2024.

[2] Sougandh, Thatavarthi Giri, Nithish Sagar Reddy and Meena Belwal, Automated Resume Parsing: A Natural Language Processing Approach, In 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), IEEE, pp. 1-6, 2023.

[3] Maiia Bocharova, Eugene Malakhov and Vitaliy Mezhuyev, VacancySBERT: Representation of Titles and Skills for Semantic Similarity Search, arXiv preprint arXiv:2307.16638, 2023.

[4] Marcel Naudé, Kolawole John Adebayo and Rohan Nanda, A Machine Learning Approach to Detecting Fraudulent Job Types, AI & Society, Vol. 38, No. 2, pp. 1013-1024, 2023.

[5] Ramakrishnan Vivek, Enhancing Diversity and Reducing Bias in Recruitment through AI, Informatics. Economics. Management, Vol. 2, No. 4, pp. 0101-0118, 2023.

[6] Mashitah Mohd Tobroni and Isredza Rahmi A. Hamid, Secured Recruitment Management System with Data Retention Approach, Applied Information Technology and Computer Science, Vol. 4, No. 2, pp. 230-247, 2023.

[7] Akshata Upadhye, Illuminating HR Insights: Unraveling Patterns through Data Analytics and Visualization, Journal ID 2811, pp. 6201, 2023.

[8] Lamiaa Mostafa and Sara Beshir, Job Candidate Rank Approach Using Machine Learning Techniques, In International Conference on Advanced Machine Learning Technologies and Applications, Springer, Cham, pp. 225-233, 2021.

[9] P. Bhandari, S. Kumar and A. Singh, Resume Screening Using Hybrid Deep Learning Model, in Proc. Int. Conf. on Machine Learning and Smart Education (ICMSLE), Springer, Singapore, pp. 89-98, 2024.

[10] J. Tan, An Automated Hybrid Resume NER Based on Rule-Based and Transformer Approaches, GitHub Academic, 2024.